HACK MATRIX HACKATHON

Problem- HM10

- **PROJECT TITLE : CodeMind– Intelligent Debugging Copilot with Thinking Pattern Detection**
- **TEAM NAME : Ctrl Alt Elite**
- **TEAM MEMBERS :**
  1. **Yathartha Rastogi (24BCE10410)**
  2. **Harsh (24BSA10054)**
  3. **Roshan Gayakawad (24BCG10086)**
  4. **Shaili Berchha(24BAC10064)**

PROBLEM STATEMENT

- **The problem**

  Current debugging tools restrict beginner programmers because they only deliver solutions. This situation leads to multiple errors which result in limited knowledge and users needing outside assistance.

- **Who is affected by this problem .**
  a. Students learning programming
  b. Beginner developers
  c. Competitive coders
  d. Self-taught programmers

- **Why this problem is important ?**
  a. Slows down learning process
  b. Creates frustration and low confidence
  c. Leads to repeated mistakes
  d. Reduces coding efficiency

EXISTING SOLUTIONS AND GAPS

- **List of existing Solutions (if any)**
  a) Stack Overflow
  b) ChatGPT / AI tools
  c) Compiler error messages
  d) Online tutorials

- **Their Limitations**
  a) Generic explanations (not personalized)
  b) No tracking of user mistakes
  c) No longitudinal tracking of user error patterns
  d) Learning is passive, not adaptive
  e) Complex solutions rather than basic explanation

- **The gap our project will address:**
  - There is no system that understands how a user thinks, tracks their mistakes over time, and actively prevents future errors.
  - Our project aims to help the user grasp the basics of the program they are creating so that they don't need to use different tools for basic problems and enable them to think like a programmer

PROPOSED SOLUTION

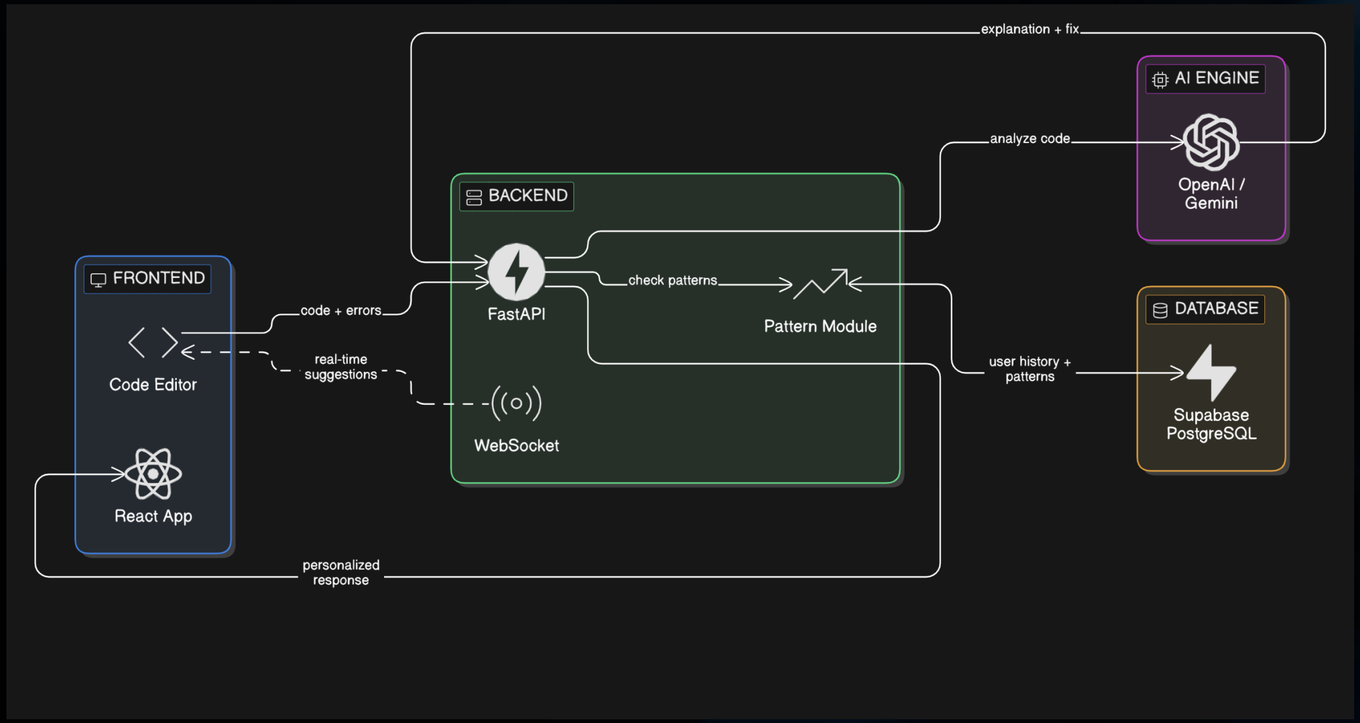
- **Our solution:**

a) An **AI-powered debugging copilot** that not only explains errors but also:
   - Tracks user mistake patterns
   - Predicts future errors
   - Adapts explanations based on user level
   - Helps the user strengthen their grasp on a programming language

- **Key features:**
  - Cognitive Error Pattern Modeling
  - Error Prediction System
  - Personalized Learning Mode
  - Error Simulation
  - Fix vs Learn Mode
  - User Coding Profile

- **How it solves the problem**
  - Converts debugging into a **learning process**
  - Prevents repeated mistakes
  - Improves coding skills faster
  - Provides personalized guidance

SYSTEM ARCHITECTURE



- **Main components**
  a) **Frontend:** React
  b) **Backend:** FastAPI
  c) **Database:** Supabase (PostgreSQL + Auth + Storage)
  d) **AI Engine:** LLM API (Gemini)
  e) **Pattern Analysis Module:** Custom logic to track user mistakes

- **Data flow**

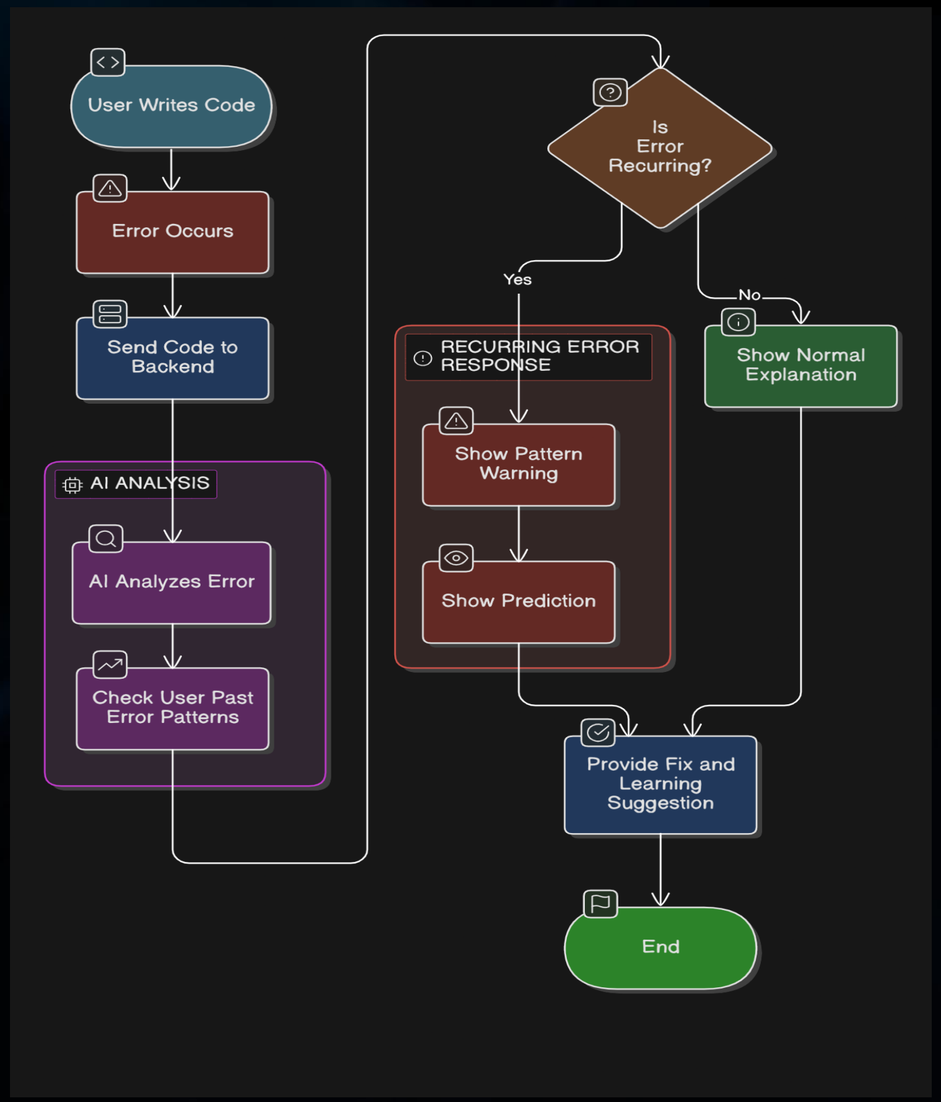
User writes code → sent to backend → backend sends to LLM API → AI returns explanation → pattern module analyses past errors → Supabase stores/updates user data → response sent back to frontend in real-time

FLOWCHART / WORKING PROCESS

- **Step-by-step working**
  1. User writes code
  2. Error detected
  3. Code sent to backend
  4. AI analyses error
  5. Pattern system checks history
  6. Prediction + explanation generated
  7. Result shown to user

- **User flow or system flow**

User → Code Input → Error Detection → AI Processing → Pattern Analysis → Output



TECH STACK

- **Technologies:**
  1. Frontend: React
  2. Backend: FastAPI
  3. Database: Supabase

- **Tools and APIs used:**
  1. Gemini API → Error explanation

IMPLEMENTATION PLAN

- **Step-by-step development plan**

Phase 1:
1. Setup frontend editor
2. Create backend API
3. Integrate AI API
4. Build error detection module

Phase 2:
1. Add pattern tracking system
2. Connect database

Phase 3:
1. Add UI features
2. Testing & deployment

- **Modules or features to be built**

1. Error Analyzer
2. AI Response Engine
3. Pattern Tracker
4. User Profile System

EXPECTED OUTCOME / IMPACT

- **Expected results of the project**
  1. Faster debugging
  2. Better learning
  3. Reduced repeated errors
- **Benefits to users**
  1. Personalized coding help
  2. Improved confidence
  3. Efficient learning
- **Real-world impact**
  1. Helps millions of beginner developers
  2. Can be integrated into IDEs
  3. Improves coding education

FUTURE SCOPE AND CONCLUSION

- **Future improvements**

  1. Multi-language support
  2. IDE plugin (VS Code extension)
  3. Bookmark & Revisit Mistakes
  4. Team collaboration features

- **Final summary**

  CodeMind transforms debugging from a **fixing process into a learning system**

- **Scalability of the project**

  1. Cloud-based architecture
  2. Can scale to millions of users
  3. Extendable to enterprise developer tools